Reproducing Results on the Andrews–Curtis Conjecture

Classical Search and PPO

AC-Solver Project

April 2026

Outline

Introduction

The Andrews–Curtis Conjecture

Conjecture (Andrews & Curtis, 1965)
Every balanced presentation of the trivial group can be reduced to $\langle x, y \mid x, y \rangle$ using a finite sequence of **AC moves**.

Computationally, this is a **graph search problem**:

- **Node:** a balanced presentation $\langle x, y \mid r_0, r_1 \rangle$
- **Edge:** one of 12 AC moves
- **Goal:** reach a trivial node (total relator length $= 2$)

Status: open for 60+ years despite extensive computational effort.

The Miller–Schupp Dataset

Parameterised by integer $n \geq 1$ and a word w in generators $\{x, x^{-1}, y, y^{-1}\}$ with **zero exponent sum on** x (i.e. $\#x = \#x^{-1}$ in w):

$$\mathrm{MS}(n, w) \;=\; \langle x, y \mid \underbrace{x^{-1}\, y^{n}\, x\, y^{-(n+1)}}_{r_0\text{: fixed by } n},\quad \underbrace{x^{-1}\, w}_{r_1\text{: varies with } w} \rangle$$

- n controls the **complexity** of the first relator r_0
- w is a **balanced word** that defines the second relator $r_1 = x^{-1}\, w$
- $n \in \{1, \ldots, 7\}$, $|w| \in \{1, \ldots, 7\}$; after removing cyclic permutations: **1190 presentations**

Example 1 (n=1, w=y, $|w|$=1): instance #0 in the dataset

$\langle x, y \mid x^{-1}\, y\, x\, y^{-2},\ x^{-1}\, y \rangle \;\longleftrightarrow\;$ `[-1, 2, 1, -2, -2, 0,..., -1, 2, 0,...]`

Example 2 (n=2, w=y, $|w|$=1): instance #170

The 12 AC Moves

The 12 AC Moves

4 Concatenations

ID	Operation
0	$r_1 \leftarrow r_1 \cdot r_0$
1	$r_0 \leftarrow r_0 \cdot r_1^{-1}$
2	$r_1 \leftarrow r_1 \cdot r_0^{-1}$
3	$r_0 \leftarrow r_0 \cdot r_1$

8 Conjugations

ID	Operation
4	$r_1 \leftarrow x^{-1} \cdot r_1 \cdot x$
5	$r_0 \leftarrow y^{-1} \cdot r_0 \cdot y$
6	$r_1 \leftarrow y^{-1} \cdot r_1 \cdot y$
7	$r_0 \leftarrow x \cdot r_0 \cdot x^{-1}$
8	$r_1 \leftarrow x \cdot r_1 \cdot x^{-1}$
9	$r_0 \leftarrow y \cdot r_0 \cdot y^{-1}$
10	$r_1 \leftarrow y \cdot r_1 \cdot y^{-1}$
11	$r_0 \leftarrow x^{-1} \cdot r_0 \cdot x$

- **Free reduction** applied after every move ($x \cdot x^{-1} \to \epsilon$)
- Move **rejected** if result exceeds `max_relator_length` $= 36$ (no-op)
- $\sim$69% of PPO actions are no-ops

Classical Search

Greedy Search

Core idea: always expand the state with the shortest total length.

Algorithm:

1. Push initial state onto a **min-heap** (priority = total relator length)
2. Pop shortest state; try all 12 moves
3. If new length = 2: solved!
4. If unseen: push onto heap
5. Repeat until solved or node budget exhausted

`ac_solver/search/greedy.py`

Properties:

- ✓ Fast (explores fewer nodes)
- × No shortest-path guarantee
- Tie-break: path length, then lexicographic
- Cost: $O(\log n)$ per step

Breadth-First Search (BFS)

Core idea: expand states level by level (all depth d before $d+1$).

Algorithm:

1. Push initial state onto a **FIFO queue**
2. Pop oldest state; try all 12 moves
3. If new length $= 2$: solved! (guaranteed shortest path)
4. If unseen: append to back of queue
5. Repeat until solved or node budget exhausted

`ac_solver/search/breadth_first.py`

Properties:

- ✓ **Shortest-path guarantee**
- × Slower (exhaustive per level)
- Cost: $O(1)$ per step

First solution found is always optimal (fewest AC moves).

Greedy vs. BFS

Aspect	Greedy	BFS
Data structure	Min-heap	FIFO deque
Expansion order	Shortest length first	By depth
Shortest path?	No	Yes
Typical speed	Faster	Slower
Per-step cost	$O(\log n)$	$O(1)$

Trade-off: Greedy is biased toward short states (fast, approximate). BFS is unbiased (slow, optimal).

Classical Search Pipeline: No Training Required

Dataset: all 1190 Miller–Schupp presentations ($n \in [1, 7]$, $|w| \in [1, 7]$).

Classical (Greedy / BFS):

1. Load dataset (1190 presentations)
2. For each: run search ($\leq 10^6$ nodes)
3. Record solved / path / time to CSV
4. No model, no training, no GPU

Multiprocessing + resumable CSV.

Wall-clock: $\sim$2 h (greedy) / $\sim$6 h (BFS).

PPO (contrast):

1. Load dataset
2. Train policy for 10^8 steps ($\sim$60 h)
3. Load best checkpoint
4. Evaluate on all presentations

Requires GPU/MPS, hyperparameter tuning, and careful reward engineering.

`ac_solver/search/miller_schupp/evaluate.py`

Classical Search Results

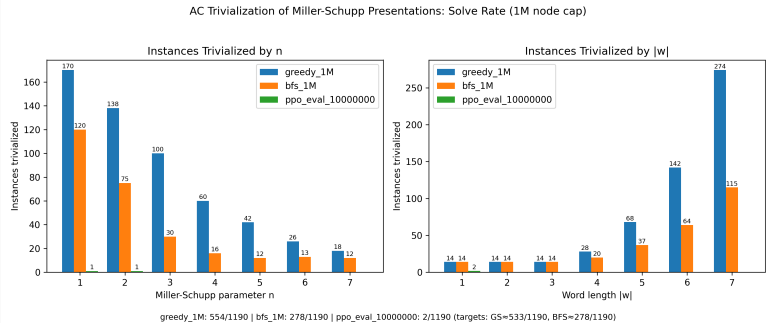
Algorithm	Solved	Solve Rate	Paper
Greedy (10^6 nodes)	**554** / 1190	**46.6%**	533 (44.8%)
BFS (10^6 nodes)	278 / 1190	23.4%	278 (23.4%)

- BFS matches the paper **exactly**
- Greedy: +21 instances (minor tie-breaking differences)

Reproduce:

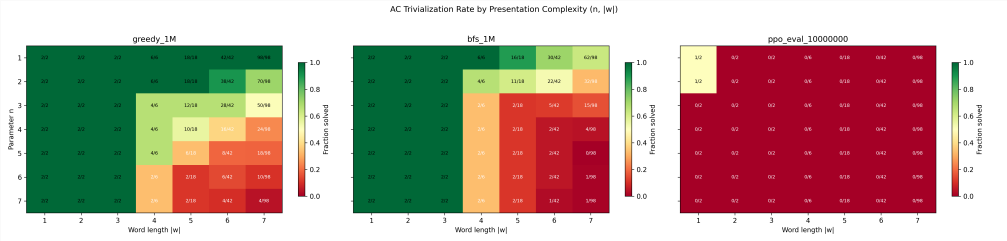
```
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 --workers 4 --full
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn bfs --max-nodes 1000000 --workers 4 --full
```

Results by Difficulty: Solve Rate



- **Left:** solved vs. n. Higher $n \Rightarrow$ longer $r_0 \Rightarrow$ larger search space. Both greedy and BFS degrade sharply.
- **Right:** solved vs. $|w|$. Weaker effect—greedy solves *more* at high $|w|$ (free-reduction opportunities).
- **PPO** (green) solves almost nothing at any difficulty level.

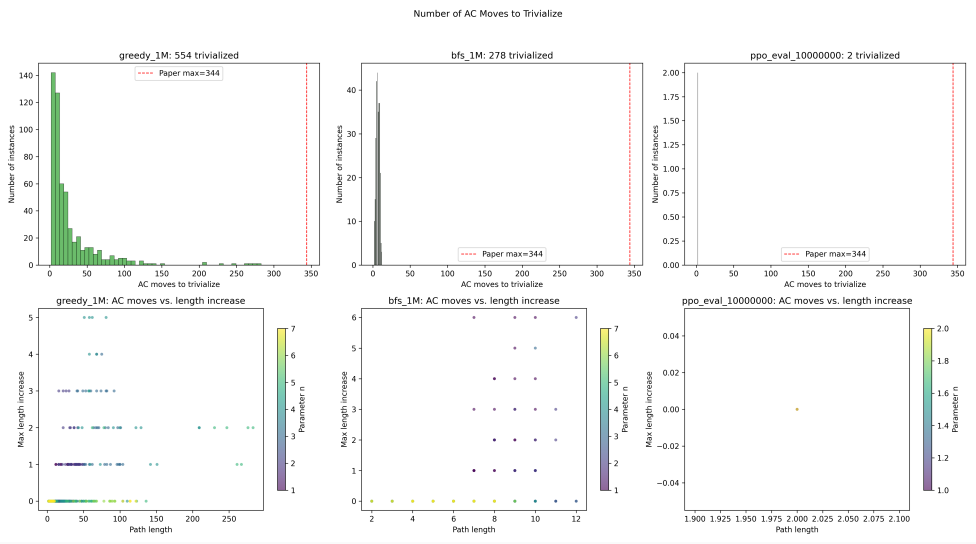
Results by Difficulty: Heatmap



Three heatmaps showing solve rate for each $(n, |w|)$ cell (green = high, red = low):

- **Greedy (left):** strong at low n (top rows), degrades toward bottom-left (high n, short w).
- **BFS (centre):** similar pattern but solves fewer cells overall; its exhaustive search hits the 10^6-node budget sooner.
- **PPO (right):** almost entirely red—the policy cannot solve any cell reliably.

Path Lengths and Length Increase



PPO Reproduction

PPO Agent Architecture

Network	Architecture	Init
Actor	$72 \to 512 \to 512 \to 12$ (Tanh)	Ortho ($\sqrt{2}$ / 0.01)
Critic	$72 \to 512 \to 512 \to 1$ (Tanh)	Ortho ($\sqrt{2}$ / 1.0)

- **Input:** $72 = 2 \times 36$ (two relators, zero-padded)
- **Output:** 12 logits $\to$ $\mathrm{Categorical}$ (training) / arg max (eval)
- Separate actor and critic (no shared layers)
- Actor output init $= 0.01 \Rightarrow$ near-uniform initial policy

Hyperparameters: Paper vs. Mac Reproduction

Parameter	Paper	Mac	Notes
`total_timesteps`	10^8	10^8	Same
`num_envs`	28	8	Hardware
`batch_size`	5,600	1,600	3.5× smaller
`update_epochs`	1	1	Single pass
`learning_rate`	$10^{-4} \to 0$		Linear decay
γ	0.999	0.999	
ϵ_{clip}	0.2	0.2	
c_{ent}	0.01	0.01	
PPO updates	17,857	62,500	3.5× more

3.5× more updates, but each with 3.5× noisier batch.

Paper vs. Reproduction: What Matches, What Doesn't

Aspect	Paper	Our Repro	Match?
Network	$[512, 512]$ MLP, Tanh	Same	✓
Separate actor/critic	Yes	Yes	✓
Weight init	Orthogonal	Orthogonal	✓
LR schedule	$10^{-4} \to 0$ linear	Same	✓
PPO hyperparams	Table 1 (Appendix A)	All identical	✓
Adam ϵ	10^{-5}	10^{-5}	✓
Curriculum	$\frac{1}{4}$ solved, $\frac{3}{4}$ unsolved	Same	✓
AC moves	12 (AC'1 + AC'2)	12 (same set)	✓
`max_relator_length`	36	36	✓
Parallel actors	28	8	×
Batch size	5,600	1,600	×
Reward normalisation	**None**	$\div 14{,}400$	×
Variable horizon	Yes (200→1200)	No (200 only)	×
PPO solved	**431 / 1100 (39.2%)**	**39 / 1100 (3.5%)**	**×**

The Reward Mismatch: Smoking Gun

The paper defines the reward function **directly** (Section 4.3):

$$R(s_t, a_t, s_{t+1}) = \begin{cases} -\min(10,\ \text{length}(s_{t+1})) & \text{if } \text{length}(s_{t+1}) > 2 \\ +1000 & \text{otherwise (solved)} \end{cases}$$

No further normalisation is applied. The PPO update sees rewards in $[-10, +1000]$.

Our code adds an extra step not present in the paper:

$$\text{rewards} \ /= \ \texttt{max\_reward} = 14{,}400 \quad \Longrightarrow \quad \text{rewards} \in [-0.0007,\ +0.069]$$

	Paper	**Our code**
Success reward seen by PPO	$+1000$	$+0.069$
Per-step penalty	-10	-0.0007
Signal-to-noise	High	$14{,}400\times$ weaker

Other Differences and What May Be Missing

Batch size (3.5× smaller)

- 8 envs × 200 steps = 1,600
- Paper: 28 × 200 = 5,600
- Noisier gradient estimates
- Fewer diverse presentations per batch
- Partially compensated by 3.5× more updates

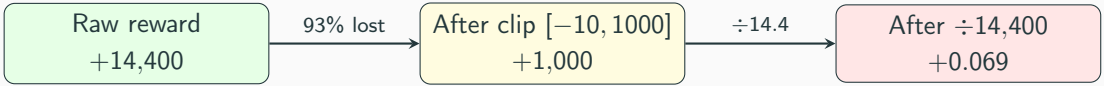
Variable horizon (not reproduced)

- Paper: $T = 200 \rightarrow 400 \rightarrow 800 \rightarrow 1200$
- Increased at 10M, 25M, 50M steps
- Solved 2 new presentations greedy couldn't
- We only used constant $T = 200$
- Limits ability to solve long-path instances

Priority Fix List

1. **Remove** `rewards /= max_reward` — restore paper's reward scale
2. **Increase** `num_envs` to 28 (or max hardware allows)
3. **Implement** variable horizon schedule

Reward Pipeline: The 200× Attenuation Problem



- **Raw success:** $+14{,}400$ $(= 200 \times 36 \times 2)$
- **Clipped:** $+1{,}000$ $\Rightarrow$ 93% of signal discarded
- **Normalised:** $+0.069$ $\Rightarrow$ near noise level
- Per-step penalty: -0.00069

This is the single biggest issue with the current training setup.

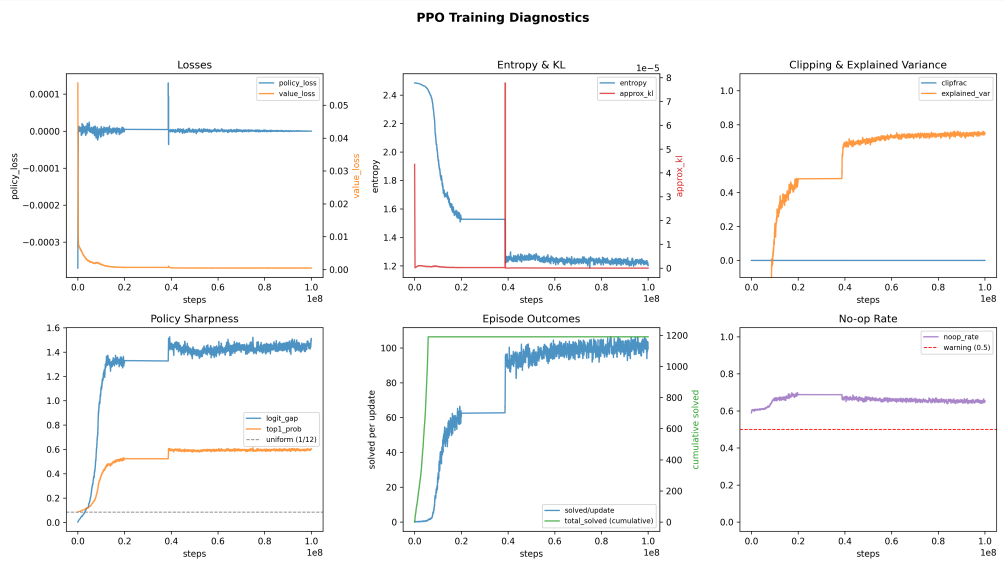
Training Execution

Phase	Steps	Updates	Duration
Initial	$0 \rightarrow 38.8$M	$1 \rightarrow 24{,}285$	$\sim$24 h
Resumed	$38.7\text{M} \rightarrow 100\text{M}$	$24{,}201 \rightarrow 62{,}500$	$\sim$36 h
Total	10^8	**62,500**	**$\sim$60 h**

Checkpoints saved at: 1M, 5M, 10M, 20M, 50M, 100M steps.

Curriculum: cycle through 1190 presentations, then revisit solved with $p = 0.25$.

Training Curves



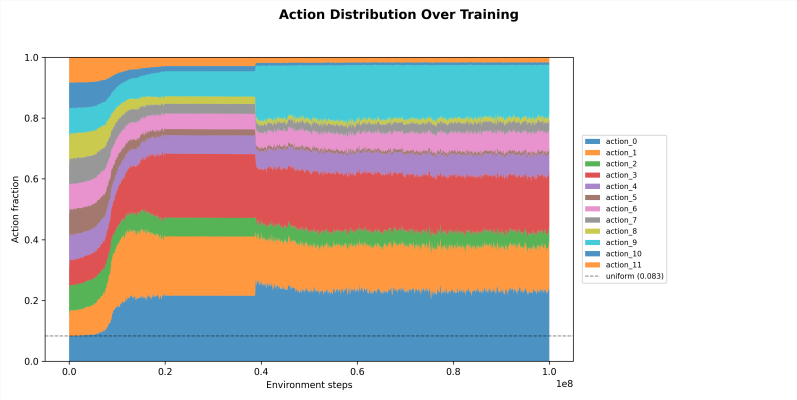
Evidence of Learning

Four metrics confirm the network changed significantly:

Metric	Start	End (100M)	Interpretation
Entropy	2.485	1.253	Policy concentrates on subset
Top-1 prob	8.4%	60.1%	Strong action preference
Logit gap	0.003	1.415	Distinct preferences
Expl. var	-0.92	$+0.66$	Critic predicts success
Clip frac	0.0	0.0	Expected with epochs $= 1$

clipfrac $= 0$ is expected, not a bug—single epoch + tiny normalised advantages $\Rightarrow$ ratio stays within $[0.8, 1.2]$.

Action Distribution Shift



Policy shifts from uniform (∼8.3% each) to concentrated—favours **concatenations** (actions 0–3) over conjugations.

PPO Evaluation Results

Checkpoint	Greedy (det)	Stoch $\times 1$	Stoch $\times 5$
1M	0 (0.0%)	2 (0.2%)	7 (0.6%)
5M	5 (0.4%)	6 (0.5%)	15 (1.3%)
10M	7 (0.6%)	12 (1.0%)	30 (2.5%)
20M	7 (0.6%)	10 (0.8%)	30 (2.5%)
50M	**6 (0.5%)**	**16 (1.3%)**	**32 (2.7%)**
100M	6 (0.5%)	13 (1.1%)	29 (2.4%)

- **Peak at 50M**—LR$\rightarrow$0 schedule degraded the policy after 50M
- All 1190 instances, horizon $= 200$

Comparative Analysis

PPO vs. Classical Baselines

Algorithm	Solved	Rate
Greedy search (10^6 nodes)	**554** / 1190	**46.6%**
BFS (10^6 nodes)	278 / 1190	23.4%
PPO 50M (greedy eval)	6 / 1190	0.5%
PPO 50M (best-of-5 stoch)	32 / 1190	2.7%

Classical greedy search is $17\times$ **better** than the best PPO configuration.

Did PPO Learn Anything?

Compare best PPO (50M sto5) vs. near-random (1M sto5):

Policy	Solved	Interpretation
Near-random (1M)	7 / 1190 (0.6%)	Random walk baseline
PPO 50M	32 / 1190 (2.7%)	4.6× improvement
Overlap	4	Solved by both
PPO-only	28	Genuine learning
Random-only	3	Noise

Verdict: PPO learned *genuine but shallow* problem structure.
Solves only the easiest instances (short lengths, low n).

Training Solve Rate $\neq$ Evaluation Solve Rate

Training:

- 94.5% solve rate
- 1190/1190 marked "solved"
- Uses stochastic sampling
- 200-step random walk often finds easy solutions

Evaluation:

- 0.5% greedy solve rate
- 2.7% best-of-5 stochastic
- Tests what policy *actually learned*
- Near-random baseline: 0.6%

Training solve rate is misleading—dominated by stochastic exploration, not learned policy quality.

Diagnosis & Recommendations

Why PPO Underperformed

1. **Catastrophic reward attenuation**
   Success signal: $+14{,}400 \rightarrow +0.069$ ($200\times$ reduction)
2. **Insufficient data reuse** (`update_epochs` $= 1$)
   Standard PPO uses 3–10 epochs per batch
3. **Batch size $3.5\times$ too small**
   1,600 vs. paper's 5,600 $\Rightarrow$ noisier gradients
4. **LR$\rightarrow$0 degradation**
   50M checkpoint outperforms 100M
5. **High noop rate ($\sim$69%)**
   Two-thirds of actions produce no state change

Recommended Next Steps

1. **Fix reward scaling** — remove clip or use running normalisation
   Highest expected impact: 200× more signal
2. **Increase `update_epochs` to 4** — 4× more learning per batch
   `target_kl` = 0.01 prevents overfitting
3. **Increase `num_envs` to 16–28** — better gradient quality
4. **Cosine LR with floor** — `min_lr_frac` = 0.1 (maintain 10^{-5})
5. **Action masking** — prevent guaranteed no-op moves
6. **Monitor greedy eval during training** — ground-truth metric

Conclusions

Summary

Classical Search

- Greedy: **554/1190 (46.6%)**
- BFS: 278/1190 (23.4%)
- No training required
- BFS matches paper exactly

PPO (100M steps)

- Best: **32/1190 (2.7%)**
- 4.6× better than random
- 17× worse than greedy
- Code correct; signal too weak

Key Takeaway

The PPO implementation is mechanically correct and the policy learned genuine (but shallow) structure. The primary bottleneck is reward signal attenuation (200×), not a bug. Classical search remains far superior.

Appendix: Reproduction Commands

Classical search:

```
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 --workers 4 --full
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn bfs --max-nodes 1000000 --workers 4 --full
```

PPO training:

```
python -m ac_solver.agents.ppo --preset paper_faithful_mac --seed 1
```

PPO evaluation:

```
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run>/ckpt_50000000.pt \
    --horizon 200 --full --stochastic --num-rollouts 5
```

Appendix: Full Hyperparameter Table

Parameter	Paper	Mac	Notes
`total_timesteps`	10^8	10^8	Same
`num_envs`	28	8	Hardware
`num_steps`	200	200	Episode horizon
`batch_size`	5,600	1,600	envs $\times$ steps
`minibatch_size`	1,400	400	batch / 4
`update_epochs`	1	1	
LR	$10^{-4} \to 0$		Linear
γ / λ_{GAE}	0.999 / 0.95	0.999 / 0.95	
ϵ_{clip}	0.2	0.2	
c_{ent} / c_{vf}	0.01 / 0.5	0.01 / 0.5	
`max_grad_norm`	0.5	0.5	
KL target	0.01	0.01	Early stop
`repeat_solved_prob`	0.25	0.25	Curriculum
PPO updates	17,857	62,500	

Appendix: Metric Trajectory

Step	Entropy	Logit Gap	Top-1	Expl. Var	Clip	Solved
1.6K (start)	2.485	0.003	0.084	−0.920	0.0	0
1M	2.483	0.035	0.093	−2.062	0.0	116
5M	2.454	0.172	0.130	−1.332	0.0	843
10M	1.919	1.162	0.397	+0.397	0.0	1190
20M	1.363	1.667	0.591	+0.510	0.0	1190
50M	∼1.2	∼1.4	∼0.58	∼0.6	0.0	1190
100M	1.253	1.415	0.601	+0.656	0.0	1190

Phase	Entropy	Top-1	Expl. Var	Noop	Solve %
0–1M	2.484	0.089	−1.832	0.604	2.5%
5–10M	2.283	0.229	−0.559	0.632	49.3%
50–100M	1.235	0.594	+0.734	0.656	94.5%